EigenDEXplore: 분석 리포트

게시용 버전: 논문 그림은 라이선스(arXiv 비독점 배포) 때문에 제외했습니다. 원문 https://arxiv.org/abs/2610.07681 · arXiv:2610.07681v2 · Stanford (Gupta, Lum, Wang, Pan, Liu, Bohg, Song) · 코드 hgupt3/eigendexplore @ 5d57180 (MIT) · 분석일 2026-10-29

핵심 요약

1. **무엇을 하는가:** 사람 손 자세를 로봇으로 리타겟한 PCA 방향을 행동 공간이 아니라 **탐색 잡음**에만 넣는다. $\Sigma = \mathrm{diag}(\sigma_q^2) + E^T\mathrm{diag}(\sigma_e^2)E$인 low-rank + 대각 Gaussian 하나이고, 정책과 배포는 그대로다. 공식 패키지는 torch `LowRankMultivariateNormal`로 이를 정확히 구현한다.
2. **수치는 맞는다:** +34%(SimToolReal), +40%(DeXtreme), 실물 33 → 82%(과제별 값으로 재계산), +41%, SPIDER –16%, DexMachina +28%(Fig. 5 판독, 과제 보상만)가 모두 재현된다. 공개 PCA 자산의 k(90%)도 여섯 손 모두 논문과 같다.
3. **공개 SimToolReal 설정은 탐색 총량이 두 배다:** 논문은 "총 분산을 Joint-Space와 맞추고 반씩"이라고 쓰지만, 공개 preset은 IID $\sigma = 1$을 그대로 두고 eigen 잡음을 더해 관절당 초기 분산이 2.0이다(스펙트럼도 λ^¼). 서술과 맞는 것은 DeXtreme뿐이고, SPIDER는 $\sigma_e \propto \lambda$에 총량도 늘어난다. 헤드라인 SimToolReal·실물 결과는 구조와 크기가 섞였을 수 있다.
4. **"EgoSuite 사전 분포"는 실제로 ARCTIC·GRAB·EgoSuite 혼합이다:** 공개 기저 manifest의 가중치는 [0.6, 0.2, 0.2]이고, 원 시퀀스는 3,596개다. fitting 파이프라인과 코퍼스가 없어 데이터셋 비교(Fig. 7a)는 재현할 수 없다.
5. **이득이 나는 조건이 분명하다:** 표본 최적화에서는 Σ가 기울기를 전처리해 사람 방향 보폭이 2.2배가 되지만, PPO에서는 조밀 보상의 기대 기울기가 Σ와 무관하다. 그래서 이득은 희소한 성공의 발견에서 나오고, 커리큘럼·보조 보상이 있으면 +6%로 줄어든다(DexMachina 판독). 자유도가 적은 손에서는 이득이 거의 없다.
6. **구현 가능성:** 방법 패키지는 바로 쓸 수 있고(MIT, 손 6종 기저 포함), CPU 예제도 있다. 세 벤치마크는 고정 커밋과 패치로 재현 환경이 갖춰져 있지만 모두 NVIDIA GPU가 필요하고, DexMachina 실험과 다른 손의 기저 만들기는 공개되지 않았다.

01. 논문 분석: EigenDEXplore (arXiv:2610.07681)

- 제목: *EigenDEXplore: Structured Exploration for Dexterous Manipulation with Human Priors*
- 저자: Harsh Gupta*, Tyler Ga Wei Lum*, Changhao Wang, Chuer Pan, C. Karen Liu, Jeannette Bohg, Shuran Song (Stanford University)
- 공개: arXiv v1 2026-10-06, **v2 2026-10-13**(분석 기준, 15쪽: 본문 9쪽 + 참고문헌 + 부록 A-B) · 코드 https://github.com/hgupt3/eigendexplore (확인 커밋 `5d57180`, 2026-10-11 "Initial release", MIT) · 프로젝트 https://eigendexplore.github.io
- 라이선스: arXiv 비독점 배포(nonexclusive-distrib 1.0)라 **CC BY가 아닙니다.** PDF 안에 다른 권리 표기는 없습니다.
- 분석 범위: v2 본문·부록 전체, v1 대조, 프로젝트 페이지, 공식 코드(패키지, 벤치마크 어댑터, 공개 PCA 자산, 고정된 외부 벤치마크 설정)
- v1 ↔ v2(문장 단위 diff): 수치·그림·표는 같습니다. 바뀐 것은 실험 조건을 밝히는 문장 네 곳입니다. (1) Sec. 1의 "early SimToolReal training, larger gains in extended runs"가 "at a fixed training budget"로 바뀌었습니다. (2) Sec. 4.1에서 "under each benchmark's protocol"이 빠지고 "We adapt the training pipeline of SimToolReal"과 "원본과 달리 Sharpa 외전 관절의 전체 범위를 쓴다"는 문장이 들어갔습니다. (3) 연장 학습은 "방법마다 실행 하나"라고 명시했습니다. (4) Sec. 4.5의 "original … hardware protocols"가 "hardware setups"로 바뀌고, "Joint-Space 정책은 공개 SimToolReal 체크포인트가 아니다"라는 문장이 추가되었습니다.

---

1. Abstract (한 줄 요약과 주장)

한 줄 요약: 사람 손 동작을 로봇 관절로 리타겟하고 PCA로 뽑은 "협응 방향(eigenvector)"을, **행동 공간이 아니라 탐색 잡음에만** 넣습니다. 정책은 그대로 관절 목표를 출력하고, 탐색 잡음만 IID 관절 잡음 + 사람 주성분 방향 잡음으로 바꿉니다(Eq. 2). 학습이 끝나면 기저는 버리고 평균 행동만 결정론적으로 실행합니다.

논문이 내세우는 주장은 다음과 같습니다(Sec. 1).

1. 사람 동작 사전 분포의 이득은 "협응 동작을 직접 명령하는 것"이 아니라 **상관된 탐색**에서 나옵니다. 행동 공간을 줄이거나(Eigen-Space) 늘리면(Eigen-Residual) 오히려 손해가 생길 수 있습니다.
2. 같은 방법이 **비구조 RL**(SimToolReal, DeXtreme), **레퍼런스 추적 RL**(DexMachina), **표본 기반 궤적 최적화**(SPIDER), **sim-to-real** 모두에서 이깁니다. 손은 여섯 종입니다.
3. 헤드라인 수치는 다음과 같습니다.
   - SimToolReal 연속 성공 +34%, DeXtreme +40%
   - 실물 SimToolReal 과제 진행 33% → 82%, 실물 DeXtreme +41%
   - DexMachina 추적 정확도 +28%(커리큘럼·보조 보상 없이)
   - SPIDER 비용 −16%(여섯 손 평균)
4. 이득은 자유도가 많은 손과 보상 설계가 적은 설정에서 가장 큽니다. 대규모 egocentric 영상(EgoSuite)으로 만든 사전 분포가 소규모 mocap(GRAB, ARCTIC)보다 낫습니다.

---

2. Method

2.1 전체 구조 (Fig. 1-2, Sec. 3)

```
사람 손 자세 (EgoSuite 영상 복원, 양손을 한쪽으로 거울 대칭해 합침)
        │  ManipTrans식 운동학 리타겟 (손별 정렬·스케일·손가락 대응)
        ▼
로봇 관절각 → PCA → 분산 90%까지의 상위 k개 방향 E ∈ R^{k×d} (관절 반범위로 정규화한 좌표)
        │                                          Sharpa k=9/22, XHand 6/12, Allegro 10/16,
Inspire 3/6
        ▼
탐색 분포  q⋆ ~ N(μ_q, Σ),   Σ = diag(σ_q²) + Eᵀ diag(σ_e²) E          (Eq. 2)
   RL(PPO):   μ_q = π_θ(o),  σ_q·σ_e 학습, E 고정, log-prob·entropy·KL은 이 분포로 계산 (Sec. 3.3)
   궤적 최적화(SPIDER): 후보 섭동만 바꾸고, 하위 10% 가중 평균으로 갱신, σ 고정 (Eq. 3, Sec. 3.4)
배포: μ_q만 실행, E와 탐색 파라미터는 버림
```

2.2 네 가지 행동·잡음 설계 (Eq. 1)

이름	표본 관절 목표 q⋆	행동 차원	논문이 보는 문제
Joint-Space	a_q + σ_q⊙η_q	d	관절을 따로 흔들어 부자연스러운 자세
Eigen-Space	Eᵀ(a_e + σ_e⊙η_e)	k	부분공간 밖을 표현할 수 없음
Eigen-Residual	a_q + σ_q⊙η_q + Eᵀ(a_e + σ_e⊙η_e)	d + k	표현은 되지만 행동 좌표가 중복됨
EigenDEXplore	a_q + σ_q⊙η_q + Eᵀ(σ_e⊙η_e)	d	행동은 그대로, 잡음만 구조화

초기화 규칙(Sec. 3.2)은 σ_e,i ∝ √λ_i입니다. 그리고 EigenDEXplore는 "관절 전체로 합한 초기 잡음 분산이 Joint-Space와 같고, 두 성분이 반씩 나누도록" σ_q와 σ_e를 다시 조정한다고 적습니다. 이 규칙이 공개 설정에서 벤치마크마다 다르게 구현되어 있다는 점이 이 분석의 핵심 발견 중 하나입니다(`04` D1-D3).

2.3 사람 사전 분포 (Sec. 3.1, 4.4)

- 데이터: EgoSuite-Open100K(Lightwheel 2026, 영상 10만 시간, 1.5만 과제)
- 비교 대상: GRAB(시퀀스 1,334개, 강체 51개), ARCTIC(시퀀스 339개, 관절 물체 11개)
- 리타겟: ManipTrans[Li 2025]의 운동학 절차를 손별로 고쳐 씁니다.
- 보존 분산 ρ = 0.9로 k를 정합니다. 이 값은 SPIDER sweep(Fig. 7b)으로 고른 것입니다.

---

3. Experiments

3.1 행동 공간과 탐색 (Sec. 4.1, Fig. 3-4, 부록 A)

벤치마크	설정	Joint-Space	Eigen-Space	Eigen-Residual	**EigenDEXplore**	출처
SimToolReal (Sharpa 손-팔, 도구 재배치)	9B frame, 허용 오차 6 cm·50° 고정, seed 5	7.9	개선	개선	**10.6** (+34%)	Fig. 3a
SimToolReal 연장	60B frame, 1 cm·8°로 조이는 커리큘럼, **방법마다 실행 1개**	4.9	이점 사라짐	12.0	**14.1**	Fig. 3b

벤치마크	설정	Joint-Space	Eigen-Space	Eigen-Residual	**EigenDEXplore**	출처
DeXtreme (Allegro 큐브 재배향, ADR)	2B frame, seed 5	8.8	부진	EigenDEXplore 조금 아래	**12.3** (+40%)	Fig. 4
부록 A (Δ 행동)	Joint-Δ, Eigen-Residual-Δ	SimToolReal에서 비슷, DeXtreme에서 크게 뒤짐				Fig. 10

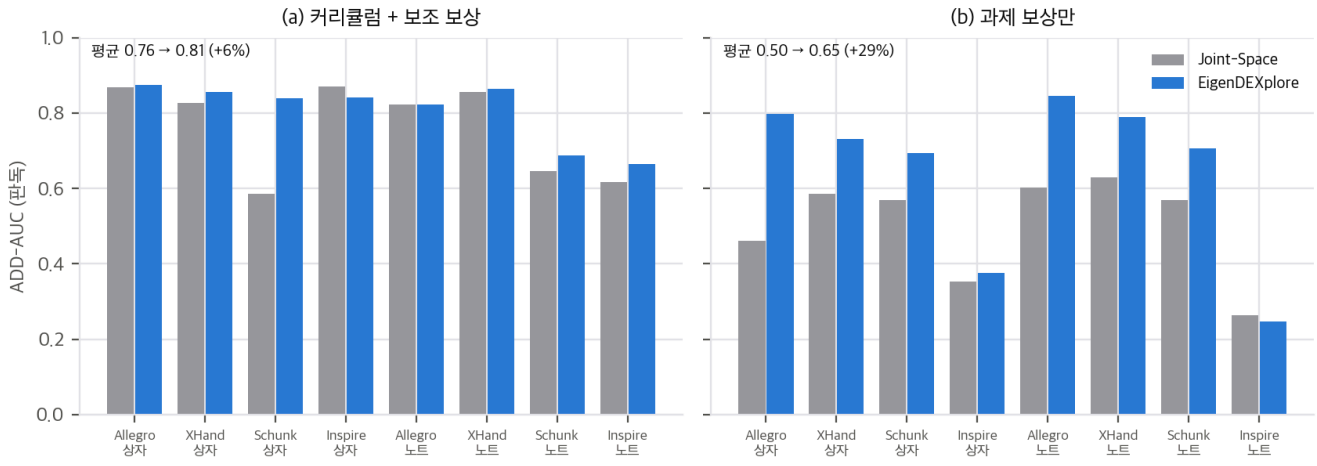
수치는 연속 성공(도달한 목표 자세 수)입니다. Eigen-Space와 Eigen-Residual의 9B 값은 그림에만 있습니다.

3.2 DexMachina: 레퍼런스 추적 양손 RL (Sec. 4.2, Fig. 5)

ARCTIC 상자·노트를 Allegro(16), XHand(12), Schunk(9), Inspire(6)로 추적합니다(seed 5, 약 2B frame, ADD-AUC). Fig. 5를 300 dpi로 판독했습니다(`03` §4, 본문 값 4개와 오차 0.004 이하).

설정	Joint-Space 평균	EigenDEXplore 평균	향상
커리큘럼 + 보조 보상	0.76	0.81	+6% (Schunk 상자 0.59 → 0.84만 큼)
과제 보상만	0.50	0.65	**+29%** (Allegro 상자 0.46 → 0.80)

본문의 "28%"는 과제 보상만 쓴 설정의 평균 비율(+28.6%)과 맞습니다. Inspire는 상자 +6.5%, 노트 −6.5%로 사실상 변화가 없습니다.



논문 Fig. 5 막대를 300 dpi에서 픽셀 판독(본문 값 4개와 오차 0.004 이하). 오차막대는 생략. analysis/fig_digitize.py

3.3 SPIDER: 표본 기반 리타겟 (Sec. 4.3, Fig. 6)

OakInk2 과제 10개, 손 여섯 종, 같은 초기 자세에서 cold start, commit당 16번 갱신, seed 10개로 비교합니다.

손 (DoF)	Sharpa (22)	Wuji (20)	Allegro (16)	XHand (12)	Schunk (9)	Inspire (6)
Δ 비용	−25.4%	−31.2%	−21.5%	−14.3%	−3.5%	−0.4%

평균은 −16.05%입니다. DoF와의 Spearman 순위 상관은 0.94입니다(`04` C6). 저자들은 "SPIDER는 잡음 크기를 학습하지 않으므로 이득은 표본 분포에서만 나온다"고 해석합니다.

3.4 데이터셋과 탐색 설계 (Sec. 4.4, Fig. 7, 부록 B)

- **데이터셋 비교(SimToolReal, 9B):** EgoSuite 10.6, ARCTIC 9.8, GRAB 9.4, Joint 7.9입니다. 그림의 95% 구간은 EgoSuite와 ARCTIC이 겹칩니다(seed 5).

- **보존 분산 sweep(SPIDER, Sharpa):**
  - Eigen 잡음만 쓰면 ρ = 0.5에서 0.126으로, Joint-Space 0.089보다 나쁩니다.
  - IID와 합치면 ρ = 0.9에서 최소 0.066입니다(Joint 대비 −25.8%).
- **무작위 방향 대조(App. B.2):** 같은 고유값 스펙트럼을 무작위 정규직교 방향에 실으면 Joint-Space보다 나쁩니다. 저자들은 이를 근거로 "상관 자체가 아니라 사람 방향이 이득의 원천"이라고 결론짓습니다.

3.5 실물 (Sec. 4.5, Fig. 8-9)

- **SimToolReal:** Sharpa + KUKA로 망치질, 닦기, 뜨기, 솔질 4과제를 합니다. 과제(물체-궤적 조합)당 10회입니다.
  - 과제별 평균(Fig. 8)은 망치질 0.42 → 0.71, 닦기 0.34 → 0.92, 뜨기 0.20 → 0.76, 솔질 0.36 → 0.90입니다. 네 과제 평균이 33% → 82%로 본문과 맞습니다(`04` C8). 정책은 방법마다 하나이고, 표준편차는 없습니다.
  - 정성 차이도 있습니다. 기준선은 가운데 손가락 패드와 검지·약지 등 사이에 도구를 끼우고, EigenDEXplore는 엄지·검지·중지 손끝으로 잡습니다.
- **DeXtreme:** 방법마다 40회입니다. 평균 연속 성공 8.45 → 11.90(+40.8%), 최장 32 → 41입니다.

---

4. Limitations

4.1 저자가 밝힌 한계 (Sec. 5)

- 선형 기저가 고정되어 있어서, 데이터에 없는 협응이나 비선형 synergy는 IID 잡음으로 찾아야 합니다.
- 사전 분포가 운동학 리타겟에 의존하므로, 사람 손과 형태가 크게 다른 손에서는 효과가 약할 수 있습니다.
- 자유도가 적은 손에서는 이득이 작습니다.
- 잡음이 시간적으로 독립입니다. 시간 상관 잡음(Raffin 2021, Eberhard 2023)을 사람 동작에서 끌어내는 것을 확장 방향으로 듭니다.

4.2 분석하면서 추가로 발견한 한계와 의문점

1. **공개 SimToolReal 설정은 탐색 분산이 기준선의 두 배입니다**(`04` D1, `03` §2).
   - 공개 preset은 IID σ를 기준선과 같은 1로 두고, eigen 잡음(RMS 1)을 그 위에 더합니다. 그래서 관절당 초기 분산이 2.0이 됩니다(Joint-Space 1.0). 스펙트럼 지수도 0.5라서 σ_e ∝ λ^¼입니다.
   - 논문 서술("총 분산을 맞추고 반씩 나눔, σ_e ∝ √λ")과 맞는 것은 DeXtreme preset뿐입니다.
   - 논문 실험이 이 preset으로 돌았다면, SimToolReal의 +34%와 실물 33 → 82%는 "구조"와 "두 배 탐색"이 섞인 결과입니다. toy에서는 탐색 크기만으로도 결과의 부호가 바뀌었습니다(`03` §3).
2. **SPIDER도 스펙트럼과 총량이 서술과 다릅니다**(`04` D2).
   - 코드 주석이 "eigenvalue power 1"이라고 밝히듯 σ_e ∝ λ입니다(√λ보다 가파름).
   - 기존 IID 잡음은 그대로 두고 eigen 잡음을 더하므로 총 분산이 늘어납니다(추정 약 1.35배).
   - 다만 SPIDER에는 같은 스펙트럼의 무작위 방향 대조군이 있어서, 크기만으로 이득을 설명할 수 없다는 근거는 이 벤치마크에서는 서 있습니다.
3. **공개된 "EgoSuite 기저"는 세 데이터셋의 가중 혼합입니다**(`04` D4).
   - 여섯 손 manifest 모두 `source_corpora`가 ARCTIC, GRAB, EgoSuite이고, `dataset_weights`가 [0.6, 0.2, 0.2]입니다(목록 순서대로 대응한다면 ARCTIC 0.6).
   - 원 시퀀스는 3,596개입니다. "10만 시간 영상"이라는 서술과 규모가 다릅니다.
   - 저장소 규칙상 기저는 하나뿐이고 fitting 파이프라인과 코퍼스가 없어서, Fig. 7a의 데이터셋 비교는 재현할 수 없습니다.

4. **SimToolReal 기준선이 원 설정보다 불리할 수 있습니다.**
    - v2에서 추가된 문장대로, 원 SimToolReal은 Sharpa의 외전 관절 범위를 좁혔습니다. 이 논문은 모든 방법에 전체 범위를 씁니다. 범위를 좁히는 것 자체가 손으로 넣은 사전 지식이라, 그것을 뺀 Joint-Space는 원래보다 약할 수 있습니다.
    - 실물 기준선도 원 공개 체크포인트가 아니라, 이 파이프라인에서 120B의 절반인 60B만 학습한 정책입니다.
    - 공개 PCA로 계산하면 정규화 좌표에서 eigen 잡음이 가장 큰 관절이 바로 외전 관절입니다(소지 MCP_AA, 엄지 CMC_AA, `03` §2).
5. **가장 큰 격차는 표본이 가장 적은 곳에서 나옵니다.** 연장 학습(4.9 대 14.1)과 실물 두 실험은 방법마다 정책 하나입니다. 9B·2B 결과도 seed 5개이고, 검정 통계는 없습니다. v1은 서론에서 "연장 실행에서 더 큰 이득"을 내세웠는데, v2는 이 문장을 "같은 학습 예산에서"로 바꿨습니다.
6. **이득은 보상 설계가 적을 때로 한정됩니다.** DexMachina에서 커리큘럼과 보조 보상이 있으면 평균 향상은 +6%이고, 한 칸(Schunk 상자)을 빼면 거의 0입니다. 정책 기울기에서 이차(조밀) 보상의 기대 기울기는 Σ와 무관하다는 점(`03` §5)과 맞는 그림입니다. 이득의 본질은 "더 나은 기울기"가 아니라 "희소한 성공의 발견"입니다.
7. **DexMachina 실험은 공개 코드에 없습니다.** README의 벤치마크는 DeXtreme, SimToolReal, SPIDER 셋뿐입니다(`04` D6).
8. **README가 권하는 시간 상관 변형은 논문 밖입니다.** AR(1), ρ = 0.94, 지속 약 16 step인 변형을 "초기 시험에서 기본보다 나을 수 있다"고 권하지만, 논문에 결과가 없습니다.

03. 핵심 식 해석과 CPU 검증

스크립트(모두 numpy, 단일 스레드, 각 수 초):

- `analysis/pca_check.py` : 공개 PCA 자산 검산, 설정별 초기 공분산 → `pca_check_results.json`
- `analysis/exploration_toy.py` : 탐색 분포 toy → `exploration_toy_results.json`
- `analysis/fig_digitize.py` : Fig. 5 판독 → `fig_digitize_results.json`
- `analysis/table_check.py` : 본문 수치 재계산 → `table_check_results.json`

공개 PCA 자산(`pca.safetensors`)은 safetensors 형식을 numpy로 직접 읽었습니다. 기저 변환은 공식 `EigenDExplore.from_pca`(반범위로 나누고 RMS를 맞춤)를 줄 단위로 옮겼습니다.

1. Eq. 2가 만드는 탐색 예산

공분산 Σ = diag(σ_q²) + Eᵀdiag(σ_e²)E에서, 정규화 관절 좌표의 관절당 평균 분산은 다음과 같습니다.

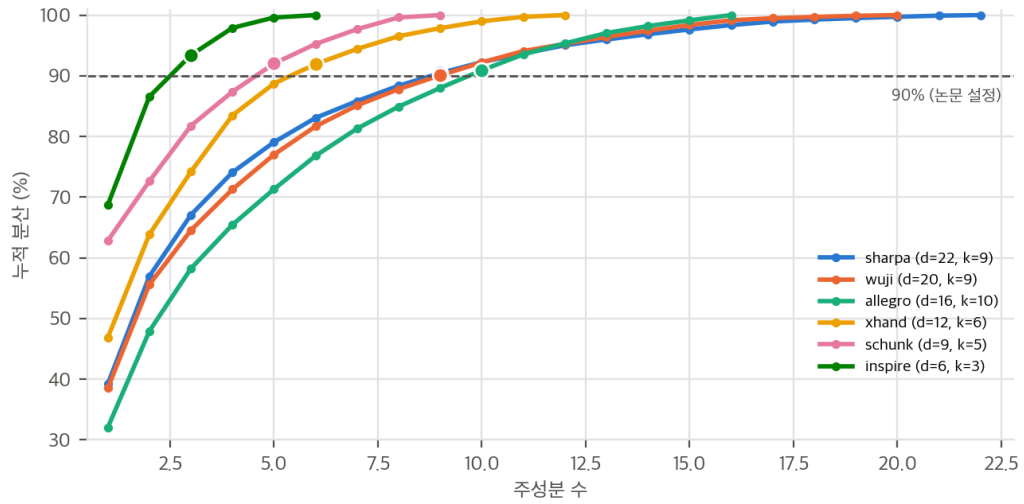
```
tr(Σ)/d = mean(σ_q²) + RMS_e²        (RMS_e: eigen 잡음의 관절당 RMS)
```

논문 서술대로 Joint-Space(σ = 1)와 총량을 맞추고 반씩 나누면 σ_q² = RMS_e² = 0.5입니다. 이때 eigen 분산은 모두 k차원 부분공간 안에 있고, IID 분산은 차원마다 고르게 퍼집니다. 그래서 부분공간 안의 분산 비율은 다음과 같습니다.

```
share_in = (0.5·d + 0.5·k) / d = 0.5 + 0.5·k/d        (등방이면 k/d)
```

손	d	k	부분공간 비율 (등방 → EigenDEXplore)	참여비 유효 차원
Sharpa	22	9	41% → **70%**	22 → 11.2
Wuji	20	9	45% → 73%	20 → 10.6
Allegro	16	10	63% → 81%	16 → 11.4
XHand	12	6	50% → 75%	12 → 6.9
Schunk	9	5	56% → 78%	9 → 5.2
Inspire	6	3	50% → 75%	6 → 3.8

공개 자산에서 보존 분산 90%에 처음 닿는 k는 여섯 손 모두 논문·catalog와 같습니다. Inspire는 k = 3에서 93.4%입니다(본문 "세 성분이 90%"와 맞음).

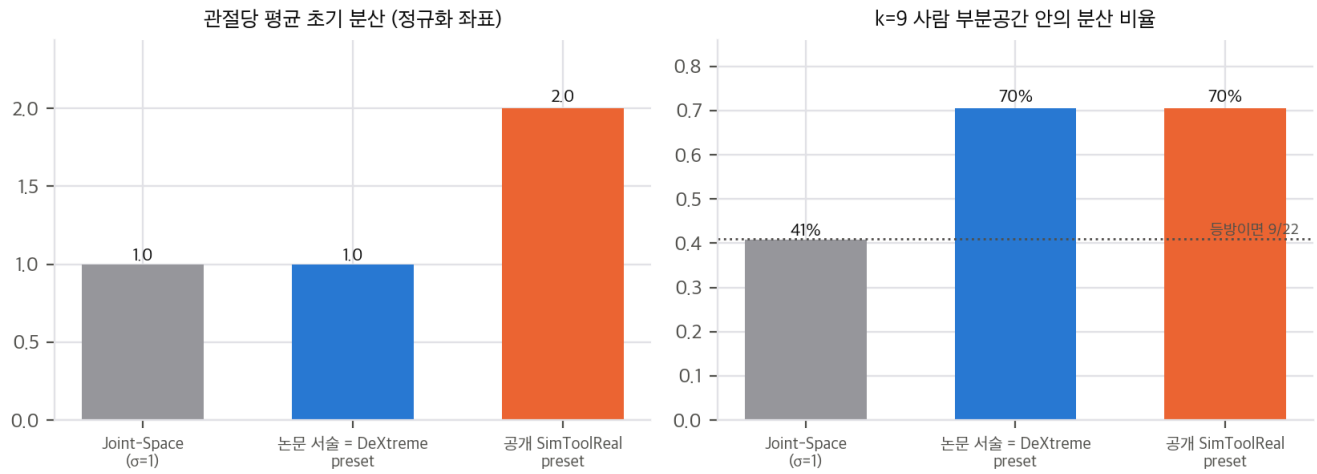


공개 PCA 자산(repo/src/action_bench/assets/*/pca.safetensors)에서 직접 계산. 점 = 90%에 처음 닿는 k. analysis/pca_check.py

2. 공개 preset은 이 예산을 지키지 않는다

설정 (Sharpa)	IID σ	eigen RMS	σ_e ∝	관절당 평균 분산	부분공간 비율
Joint-Space (원 설정 sigma_init 0)	1	-	-	1.0	41%
논문 서술 = 공개 DeXtreme preset	√0.5	√0.5	√λ	1.0	70%
공개 SimToolReal preset	1	1	λ^¼ (`spectrum_power 0.5`)	**2.0**	70%
공개 SPIDER 설정	기존 그대로	0.235 rad	**λ**	기존 대비 약 1.35배(가정)	-

- **SimToolReal:** 부분공간 집중도는 같지만 탐색 총량이 두 배입니다. 스펙트럼은 평평해서, 마지막 성분 σ_e가 첫 성분의 0.48배입니다(√λ면 0.23배).
- **SPIDER:** σ_e ∝ λ라서 마지막/첫 비가 0.053으로 훨씬 가파릅니다. 총량 1.35배는 "손가락 IID 잡음 = joint_noise_scale(0.1) × 스케줄(마지막 knot 4.0)"이라고 가정하고 계산한 값입니다.
- **정규화 좌표의 부작용:** 반범위로 나누므로 범위가 작은 관절에 정규화 잡음이 커집니다. Sharpa에서 eigen 잡음 σ가 가장 큰 관절은 소지 MCP_AA(1.01)와 엄지 CMC_AA(0.93)이고, 둘 다 반범위가 0.349 rad인 외전 관절입니다. 가장 작은 관절은 DIP와 IP(0.17)입니다. 원 SimToolReal이 좁혔던 바로 그 외전 범위를 이 논문은 넓혀서 썼습니다(`01` §4.2).



Sharpa(d=22, k=9). Joint-Space 초기 σ는 SimToolReal·DeXtreme 원 설정의 sigma_init 0(σ=1). 공개 preset 값으로 계산. analysis/pca_check.py

3. 표본 최적화(SPIDER)에서 Σ는 보폭을 바꾼다

3.1 유도

ε ~ N(0, Σ)로 평활화한 비용 J(μ) = E[f(μ+ε)]의 기울기는 ∇J = Σ⁻¹ E[f(μ+ε) ε]입니다. 그래서 다음이 성립합니다.

```
E[f(μ+ε) ε] = Σ ∇J
```

ES·CEM·MPPI 계열의 "좋은 섭동의 가중 평균"(Eq. 3)은 기대값으로 –Σ∇J 방향이고, **Σ가 기울기를 전처리(precondition)**합니다. 오차 방향 u로의 상대 보폭은 uᵀΣu / (trΣ/d)입니다. 사람 자세 분포에서 뽑은 방향과 등방 방향에서 이 값을 쟀습니다.

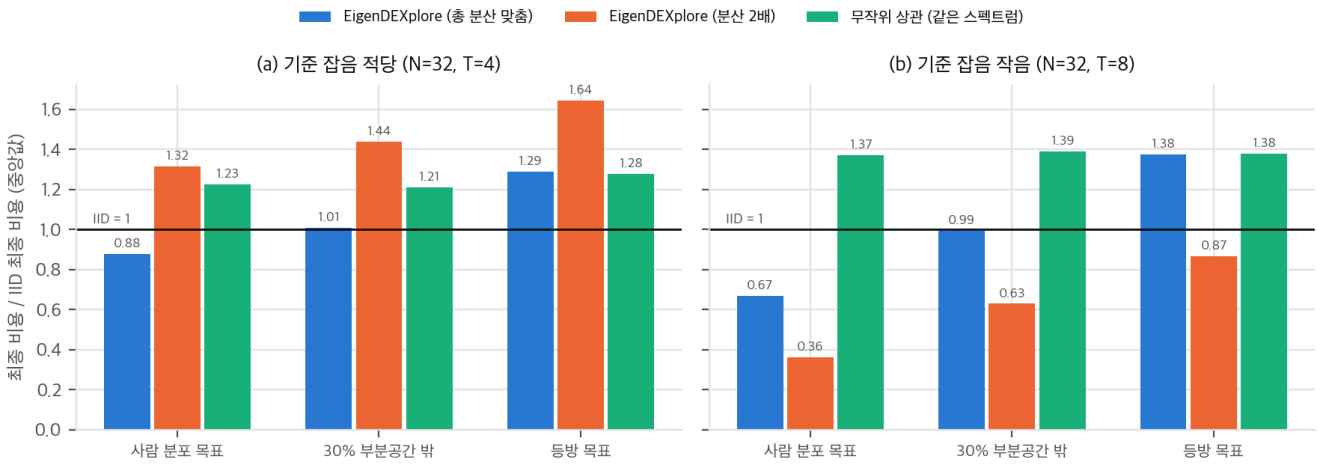
손	사람 방향 보폭	등방 방향 보폭
Sharpa	**2.21배**	1.00
Wuji	2.23배	1.00
Allegro	1.58배	1.00
XHand	1.79배	1.00
Schunk	1.90배	1.01
Inspire	1.61배	1.00

trace를 맞췄으므로 등방 방향은 평균적으로 손해가 없습니다. 다만 부분공간에 직교하는 최악 방향은 0.5배입니다.

3.2 toy (T1): 예산이 적은 SPIDER식 최적화

Sharpa 사람 공분산, 차원 22, 표본 N = 32, 하위 10% 가중 평균, 300회 쌍 비교, 수치는 IID 대비 최종 비용의 중앙값입니다.

분포	(a) 기준 잡음 적당: 사람 목표 / 부분공간 밖 30% / 등방	(b) 기준 잡음 작음: 같은 순서
EigenDEXplore (총량 맞춤)	**0.88** / 1.01 / 1.29	**0.67** / 0.99 / 1.38
EigenDEXplore (분산 2배)	1.32 / 1.44 / 1.64	**0.36** / 0.63 / 0.87
무작위 상관 (같은 스펙트럼)	1.23 / 1.21 / 1.28	1.37 / 1.39 / 1.38
Eigen만	0.94 / 1.79 / 2.56	0.95 / 2.00 / 2.92
가파른 스펙트럼 (σ_e ∝ λ)	0.97 / 1.07 / 1.34	1.00 / 1.21 / 1.56



식 3(하위 10% 가중 평균)을 numpy로 단순화. Sharpa PCA 공분산, 300회 쌍 비교. 1보다 작을수록 IID보다 낫다. 'Eigen만'은 사람 목표 0.94-0.95, 나머지 1.8-2.9배라 뺌. analysis/exploration_toy.py

해석:

- **방향이 맞을 때만 이깁니다.** 사람 분포 목표에서는 이기고(쌍 승률 64%, 86%), 등방 목표에서는 집니다. 무작위 방향은 사람 목표에서도 IID보다 나쁩니다. 논문 App. B.2와 같은 결론입니다.
- **총량 2배는 영역에 따라 부호가 뒤집힙니다.** 기준 잡음이 작은 영역에서는 구조보다 "두 배 탐색"의 효과가 더 큽니다(0.36 대 0.67, 쌍 승률 99.7%). 적당한 영역에서는 오히려 손해입니다(1.32). 따라서 탐색 총량이 다른 두 설정을 비교하면, 구조의 효과를 따로 읽을 수 없습니다. 공개 SimToolReal preset의 결과를 해석할 때 가장 중요한 점입니다.
- **Eigen만은 부분공간 밖 목표에서 실패합니다**(1.8-2.9배). 논문의 Eigen-Space 결과(DeXtreme 부진, 연장 학습에서 이점 소실)와 같은 방향입니다.

4. DexMachina Fig. 5 판독

축 눈금 5개로 픽셀을 값으로 바꾸었습니다. 본문 값과 비교한 판독 오차는 다음과 같습니다.

칸	본문	판독
Allegro 상자, 과제 보상만	0.46 / 0.80	0.460 / 0.797
Schunk 상자, 커리큘럼	0.59 / 0.84	0.586 / 0.839

8칸 평균은 커리큘럼 0.761 → 0.806(+5.9%), 과제 보상만 0.504 → 0.648(+28.6%)입니다. "28%"는 과제 보상만 쓴 설정의 평균 비율입니다. 과제 보상만 쓴 설정의 칸별 향상은 Allegro 상자 +73%, 노트 +40%, Inspire 노트 −6.5%입니다.

5. 정책 기울기(PPO)에서는 Σ가 기대 기울기를 바꾸지 않는다

REINFORCE의 기대값은 $E[R \nabla_\mu \log \pi] = E[R \Sigma^{-1}(a-\mu)] = \nabla_\mu E[R]$입니다. 이차 보상 $R = -\|a - q^*\|^2$이면 $E[R] = -\|\mu - q^*\|^2 - \mathrm{tr}\Sigma$이므로, 기대 기울기 $-2(\mu - q^*)$는 **Σ와 무관**합니다. 3절의 ES와 달리 전처리가 없습니다.

toy(T3, 배치 64, 200회)에서도 확인됩니다.

	평균 기울기 cos (참값 대비)	배치 추정 cos 중앙값
IID, 사람 목표	0.987	0.417
EigenDEXplore, 사람 목표	0.975	**0.297**
IID, 등방 목표	0.985	0.438

	평균 기울기 cos (참값 대비)	배치 추정 cos 중앙값
EigenDEXplore, 등방 목표	0.969	0.334

조밀한 이차 보상에서 구조화 잡음은 기울기를 개선하지 않습니다. 오히려 Σ^{-1}이 부분공간 밖(분산 0.5)을 2배로 키워 추정 잡음을 늘립니다.

그러면 PPO에서 이득은 어디서 오는가: 희소한 성공의 발견(T2). 목표 방향 u로 IID σ의 a배 이상 나아갈 확률 $P(\langle \varepsilon, u \rangle \geq a)$는 다음과 같습니다.

	a = 2	a = 3
IID	0.0228	0.00135
EigenDEXplore, 사람 방향	0.0855	**0.0235** (17배)
EigenDEXplore 2배, 사람 방향	0.147	0.0597 (44배)
무작위 상관, 사람 방향	0.0265	0.00255
EigenDEXplore, 등방 방향	0.0239	0.00216
EigenDEXplore 2배, 등방 방향	0.0775	0.0178 (13배)

몬테카를로로 확인한 값(a = 3, 사람 방향)은 0.0232로 해석값 0.0235와 맞습니다.

정리하면 다음과 같습니다.

- 조밀 보상에서는 기대 기울기가 같으므로 이득이 작습니다.
- 희소 보상에서는 꼬리 확률이 크게 늘어 이득이 큽니다.
- 이 구조는 논문의 관찰과 맞습니다. 커리큘럼·보조 보상이 있으면 비슷하고, 과제 보상만 있으면 크게 이깁니다.
- 꼬리 확률은 구조보다 **총량**에 훨씬 민감합니다. 등방 방향에서도 2배 분산은 13배입니다. 그래서 2절의 SimToolReal preset 차이가 이 메커니즘에서는 특히 큰 교란입니다.

6. SPIDER 이득은 무엇과 함께 커지나

Δ비용(Fig. 6)과의 Spearman 순위 상관은 다음과 같습니다(손 6개).

설명 변수	ρ	p
DoF	0.94	0.005
d − k (부분공간 밖 차원)	0.93	0.008
k (90%)	0.81	0.05
사람 방향 보폭 이득 (§3.1)	0.60	0.21

보폭 이득만으로는 Inspire(1.61배인데 −0.4%)와 Schunk(1.90배인데 −3.5%)를 설명하지 못합니다. 작은 손에서는 IID만으로도 표본 16 × 1,024개가 공간을 충분히 덮어서, 전처리의 여지가 없는 것으로 보입니다(가설). 이득이 "탐색 공간이 클 때" 나온다는 저자 해석과 맞지만, 표본 수 대비 차원으로 설명하는 것이 더 정확합니다.

추가 검증 (2026-11-11)

추가 검증 1: MuJoCo Allegro 자세 추적에서 탐색 공분산 비교 (CEM, CPU)

목적. §3.2 toy는 numpy 목적함수에서 IID, 총량 맞춤 EigenDEXplore, 총량 2배, 무작위 방향을 비교했습니다. 이 절에서는 같은 네 탐색 공분산을 MuJoCo의 Allegro 손 자세 추적에 넣어, 방향 정렬과 총량 효과가 실제 관절 동역학에서도 같은 방향으로 나타나는지 봅니다.

데이터. 손 모델은 MuJoCo Menagerie `wonik_allegro`의 `right_hand.xml`과 오른손 STL 11개(약 0.97 MB), 커밋 `370c439`(BSD-3, LICENSE 파일 확인)입니다. PCA 기저는 기존 clone의 `src/action_bench/assets/allegro/pca.safetensors`(1,776 B)를 다시 받지 않고 읽었습니다. 관절 한계는 같은 폴더의 `manifest.yaml`에서 가져왔습니다. 저장소 라이선스는 MIT입니다. 다운로드 명령은 `README.md`에 있습니다.

설정.

- 공분산은 정규화 관절 좌표(반범위로 나눔)에서 만듭니다. IID는 I, 총량 맞춤 EigenDEXplore는 0.5 I + Σ_e(PCA 스펙트럼, 90% 분산을 덮는 k = 10 성분, 대각 평균 0.5), 2배는 그 두 배, 무작위 방향은 0.5 I + 같은 스펙트럼을 무작위 직교 기저에 놓은 것입니다. 네 공분산의 trace/d는 1, 1, 2, 1입니다.
- 목표는 PCA 사전분포에서 뽑은 자세(in-subspace), PCA 부분공간 밖 방향(off-subspace), 관절 한계 안 균일 분포(isotropic)입니다.
- 최적화는 CEM입니다. 제어 4스텝(스텝당 0.1 s)의 관절 목표 4 × 16을 표본 32개, 상위 3개 엘리트, 10회 반복으로 최적화합니다. 비용은 마지막 자세의 정규화 제곱 오차 평균입니다. 시작 자세는 관절 중간값입니다.
- 탐색 보폭은 두 가지(0.25, 0.10, 정규화 단위)입니다. 같은 시드에서 네 샘플러가 같은 목표와 같은 초기 상태를 씁니다(짝 비교). 시드는 40개입니다.
- MuJoCo 4.0.0, numpy 1.26.4, 4 스레드, 총 345 s입니다.

결과. IID 대비 비용 중앙값 비와 IID보다 낮은 시드 비율(승률)입니다.

목표 · 보폭	총량 맞춤 EigenDEX	총량 2배	무작위 방향
in-subspace · 0.25	0.99 (승률 0.50)	0.97 (0.53)	1.15 (0.38)
in-subspace · 0.10	**0.89 (0.80)**	**0.52 (1.00)**	0.97 (0.62)
off-subspace · 0.25	1.14 (0.25)	1.38 (0.17)	0.99 (0.53)
off-subspace · 0.10	1.30 (0.05)	**0.77 (0.90)**	1.11 (0.38)
isotropic · 0.25	1.42 (0.30)	1.37 (0.28)	1.35 (0.28)
isotropic · 0.10	1.22 (0.23)	**0.69 (0.88)**	1.20 (0.23)

결론.

- 방향 정렬은 in-subspace 목표에서만 보입니다. 총량 맞춤 EigenDEX는 무작위 방향보다 낮습니다(0.89 대 0.97, 0.99 대 1.15). 그러나 off-subspace와 isotropic에서는 IID보다 나빠집니다(1.14-1.42). 이 점은 §3.2와 같습니다.
- 총량 2배는 보폭이 작을 때 모든 목표에서 이깁니다(0.52-0.77). isotropic 목표에서도 0.69입니다. 구조가 아니라 탐색 총량의 효과가 더 큽니다. §3.2의 "총량이 다른 두 설정은 구조 효과를 따로 읽을 수 없다"는 주의가 실제 MuJoCo 자세 추적에서도 성립합니다.
- 보폭이 큰 영역(0.25)에서는 총량 2배도 이득이 없습니다. 최적 보폭이 설정에 따라 바뀌므로, 공개 preset 비교에서 보폭을 먼저 맞춰야 합니다.

한계.

- 접촉이 없는 자세 추적입니다. 물체가 없고 자기 충돌과 관절 한계만 있습니다. 접촉 동역학에서의 결론은 이 절로 알 수 없습니다.
- 목표 비용은 정규화된 최종 자세 오차 하나입니다. 과제 성공률이나 SPIDER·DeXtreme의 보상과 연결하지 않았습니다.
- 시작 자세, 제어 스텝 수, CEM 설정(표본 32, 엘리트 3, 10회)은 하나만 봤습니다. 보폭은 두 값뿐입니다.
- 공개 PCA 기저는 16손가락 분포에서 얻은 것이고, 목표 분포는 이 기저에서 뽑은 가정입니다.

추가 검증 (2026-11-14): 물체 밀기 과제에서 탐색 공분산 비교 (MuJoCo, Allegro)

스크립트: `analysis/extra_eigen_contact.py` → `analysis/extra_eigen_contact_results.json` (CPU, 약 2분 20초, 손 OpenMP 4스레드).

목적. 앞 절의 자세 추적에는 접촉이 없었습니다. 이 절에서는 같은 손과 같은 네 가지 탐색 공분산을 쓰고, 손가락으로 구를 밀어 목표 위치에 놓는 접촉 과제로 바꿔 비교합니다.

설정(가정).

- 손: 공개 Allegro 오른손(Menagerie, 16 position 액추에이터), 밑동 고정. 공분산은 앞 절과 같은 PCA 기저와 같은 trace(평균 분산 0.5 + 0.5 고유 부분, iid 대비 총량 1배)로 만듭니다.
- 물체: 반지름 1.5 cm, 질량 0.5 kg의 자유 구를 손바닥 좌표 (0.085, 0, 0.075) m에 둡니다. 시작 자세의 손가락 끝(약 0.06 m 부근)과 닿을 수 있는 거리입니다. 중력은 끄고(탁자 없음), 손은 움직이지 않는 상태에서 손가락만 구를 밉니다.
- 목표: 시드마다 구 시작점에서 3 cm 떨어진 무작위 방향의 점. 시드 40개이며, 네 샘플러와 두 보폭에서 같은 목표를 씁니다(짝 비교).
- 최적화: CEM, 제어 4 스텝 × 16 관절, 스텝 0.1 s, 표본 32, 엘리트 3, 10회 반복. 보폭(정규화 관절 단위) 0.25와 0.10.
- 비용: 구 중심과 목표의 거리 제곱을 3 cm의 제곱으로 나눈 값. 아무것도 하지 않으면 1. 성공: 최종 오차 < 1 cm.
- 무중력, 감쇠 없음, 밀기 저항 없음. 질량 0.5 kg은 질량 0.05 kg에서 구가 평균 17 cm 날아가는 문제를 피하려고 골랐습니다(검증 탐색 과정에서 정함). 이 값은 실제 물체의 질량과 맞지 않습니다.

결과. 비용 중앙값, 아무 동작도 하지 않을 때(1.0)와 비교한 비율, iid를 이긴 시드 수(동점은 짐), 성공률, 최종 오차 중앙값, 짝 비교 Wilcoxon p.

보폭	샘플러	비용 중앙값	iid 대비 비율	iid 이긴 수 (/40)	성공률 (< 1 cm)	최종 오차 중앙값	Wilcoxon p
0.25	iid	0.316	1	—	0.20	16.9 mm	—
0.25	eigen_total	0.314	0.986	21	0.25	16.8 mm	0.27
0.25	eigen_2x	0.418	1.065	11	0.15	19.4 mm	0.009
0.25	random_dir	0.297	0.988	24	0.30	16.3 mm	0.17
0.10	iid	0.282	1	—	0.35	15.9 mm	—
0.10	eigen_total	0.316	0.997	21	0.325	16.9 mm	0.90
0.10	eigen_2x	0.331	0.998	20	0.325	17.3 mm	0.46
0.10	random_dir	0.317	1.000	18	0.35	16.9 mm	0.89

동점(같은 비용) 쌍은 샘플러마다 3-4개입니다(보폭 0.25의 random_dir만 3개). 부호 검정과 Wilcoxon 검정은 동점을 뺀 쌍에서 했습니다(Wilcoxon은 scipy 기본 처리).

결론.

- 0.25 보폭에서 총량을 2배로 한 eigen_2x는 iid보다 나쁩니다(iid를 11/40만 이김, Wilcoxon p = 0.009, 비용 중앙값 +6.5%). 자세 추적 절에서 보인 "총량 2배가 이긴다"는 결과는 이 접촉 과제에서 재현되지 않았습니다(보폭 0.10에서는 차이가 없음).
- 나머지 비교(eigen_total, random_dir)는 p > 0.1로 iid와 구분되지 않습니다. 방향 정렬 효과(eigen_total 대 random_dir)도 이 설정에서는 보이지 않았습니다.
- 성공률이 낮습니다(0.15–0.35). 최종 오차 중앙값은 16 mm로 목표 거리(30 mm)의 절반 정도만 줄였습니다. CEM 4 스텝으로는 구를 목표까지 안정적으로 보내지 못하며, 이 설정은 샘플러 차이를 가릴 만큼 강하지 않습니다.

한계.

- 과제 하나(구 하나, 목표 거리 하나, 무중력, 탁자 없음, 손 고정). 실제 파지나 밀기 물리를 재현하지 않았습니다.
- 질량 0.5 kg은 비현실적 값이며, 질량과 감쇠를 바꾼 민감도 분석은 하지 않았습니다.
- MPPI는 쓰지 않았습니다(CEM만). 표본 32, 반복 10으로 작은 규모입니다.
- 시드 40개로 p가 0.05 근처인 차이는 확실하지 않습니다. 보폭 두 개만 봤습니다.

04. 코드 검증: 논문 ↔ 구현 대조, 수치 재계산, 구현 가능성

- **대상:** `hgupt3/eigendexplore` @ `5d57180` (2026-10-11 "Initial release", 커밋 1개, MIT). v1(10-06)과 v2(10-13) 사이에 공개되었습니다.
- **함께 확인한 것:** 벤치마크가 고정해 쓰는 외부 설정을 `gh api`로 읽었습니다.
  - IsaacGymEnvs `aeed298`의 DeXtreme PPO 설정
  - `hgupt3/simtoolreal` `fddff74`의 SimToolReal PPO 설정
- **범위:** GPU 실행 없이 코드 읽기, 공개 PCA 자산 계산, CPU 재계산과 toy만 했습니다. 학습 결과·체크포인트·로그는 저장소에 없습니다.

1. 모듈 매핑

논문	구현 위치	일치 여부
Eq. 2 $\Sigma = \mathrm{diag}(\sigma_q^2) + E^T\mathrm{diag}(\sigma_e^2)E$	`packages/eigendexplore/src/eigendexplore/distribution.py` : `LowRankMultivariateNormal(mean, B^T·σ_e, σ_q²)`	✅ 정확히 같음. log-prob·entropy·KL(`policy_kl`)도 이 분포로 계산
σ_q, σ_e 학습, E 고정 (Sec. 3.3)	`log_scale` 파라미터(eigen), 호스트 logstd(IID), `basis`는 buffer	✅
관절 반범위 정규화 좌표	`from_pca(..., coordinate_scale=half_range)` : 기저를 반범위로 나눔	✅
$\sigma_{e,i} \propto \sqrt{\lambda_i}$ 초기화	`spectrum_power` 인자(component_std의 거듭제곱)	⚠️ 벤치마크마다 다름(D1, D2)
총 분산을 Joint-Space와 맞추고 반씩	`eigen_rms`, `iid_sigma` 인자	⚠️ DeXtreme과 README 기본값만 일치(D1)
90% 보존 분산 k	`assets/catalog.yaml` (k: Sharpa 9, Wuji 9, Allegro 10, XHand 6, Schunk 5, Inspire 3), `spider/rank.py`	✅ 공개 자산으로 다시 계산해도 같음(`03` §1)
양손 거울 합치기	manifest `construction: mirrored_pair`	✅
rl_games PPO 통합	`eigendexplore/rl_games.py`, `third_party/patches/*-rl-games.patch` (eigen σ를 버퍼에 저장해 KL 계산)	✅
SPIDER: 후보 섭동만 교체, 하위 10% 가중 평균, σ 고정 (Eq. 3)	`benchmarks/spider/proposal.py` (SPIDER 기본 IID 표본에 eigen 섭동 더함), `sampling.py`	✅ 구조 일치. 스케일은 D2
무작위 방향 대조 (App. B.2)	`sampling.random_orthonormal_rows` (고정 seed 20260820), `--set basis=random`	✅ SPIDER에만 있음(D8)
네 가지 행동 공간 + Δ 변형 (Eq. 1, 4)	`action_spaces/runtime.py`, `settings.py`	⚠️ 세부가 다름(D7)

논문	구현 위치	일치 여부
벤치마크 3종 + 손 6종	README: DeXtreme(Allegro), SimToolReal(Sharpa), SPIDER(여섯 손)	⚠️ DexMachina 없음(D6)

2. 불일치와 논문에 적히지 않은 구현 요소

#	내용	영향
D1	**SimToolReal preset의 탐색 총량이 두 배입니다.** `presets/simtoolreal.yaml`은 `iid_sigma: 1.0, eigen_rms: 1.0, spectrum_power: 0.5` 이고, 호스트 초기 σ는 1(`sigma_init val 0`)입니다. 그래서 IID 부분은 Joint-Space와 같고, 그 위에 eigen 잡음이 더해져 관절당 초기 분산이 2.0이 됩니다(Joint-Space 1.0). 스펙트럼도 σ_e ∝ λ^¼로 평평합니다. 반면 `presets/dextreme.yaml` (√0.5, √0.5, 1)과 README 기본값은 논문 서술(총량 맞춤, 반씩, √λ)과 같습니다. 팔 7채널은 IID σ 1이고 eigen 잡음이 없습니다.	논문 실험이 이 preset으로 돌았다면 SimToolReal의 +34%, 연장 학습, 실물 33 → 82%는 "구조"와 "두 배 탐색"이 섞인 결과입니다. toy에서 총량 차이는 결과의 부호까지 바꿨습니다(`03` §3.2, §5). 논문은 이 차이를 언급하지 않습니다.
D2	**SPIDER의 스펙트럼은 σ_e ∝ λ입니다.** `sampling._powered_component_scale`의 주석이 "Spider's recorded convention uses eigenvalue power 1 (not std power 1)"이라고 씁니다. eigen RMS는 손별 "ARCTIC posture PCA의 full-rank joint RMS"(Sharpa 0.235 rad, `configs/hands.yaml`)에 맞춥니다. 기존 IID는 `iid_scale 1.0`으로 그대로 두므로 총 분산이 늘어납니다(가정을 두고 약 1.35배, `03` §2). cold start 자세도 ARCTIC PCA 평균이고, 두 방법이 같이 씁니다.	서술(√λ, 총량 맞춤)과 다릅니다. 다만 SPIDER에는 같은 스펙트럼의 무작위 방향 대조군이 있어서, "크기만으로는 이득이 안 난다"는 근거가 이 벤치마크에서는 성립합니다.
D3	벤치마크마다 스펙트럼 지수가 다릅니다. DeXtreme √λ(마지막/첫 σ 비 0.23), SimToolReal λ^¼(0.48), SPIDER λ(0.053). 같은 "EigenDEXplore"라는 이름 아래 세 가지 분포가 쓰였습니다.	데이터셋 비교(Fig. 7a, SimToolReal)와 ρ sweep(Fig. 7b, SPIDER)이 서로 다른 분포 위에서 이루어졌습니다. "90% 설정"이 다른 벤치마크로 그대로 옮겨진다는 근거가 약합니다.
D4	**공개된 "EgoSuite 기저"는 세 데이터셋의 가중 혼합입니다.** 여섯 손 manifest 모두 `source_corpora`가 [ARCTIC, GRAB, EgoSuite], `dataset_weights`가 [0.6, 0.2, 0.2]입니다(목록 순서대로 대응한다고 가정). 원 시퀀스는 3,596개이고(fit 3,169개), 6 Hz, 계수 범위는 P1/P99입니다. 저장소 규칙(`AGENTS.md`)은 "손마다 fitted EgoSuite basis 하나, 원 데이터·학습 코퍼스·producer workflow는 싣지 않음, dataset selector 없음"입니다.	논문은 "EgoSuite(10만 시간) 사전 분포가 GRAB, ARCTIC보다 낫다"(Fig. 7a)고 하는데, 공개 기저는 ARCTIC 비중이 가장 큰 혼합으로 보입니다. GRAB·ARCTIC 단독 기저가 없어 Fig. 7a, App. B.1은 재현할 수 없습니다. 다른 손에 쓰려면 리타겟과 PCA fitting을 직접 해야 합니다.
D5	**SimToolReal 공개 프로토콜이 Fig. 3a와 다릅니다.** preset은 `success_tolerance: 0.075`, `curriculum_start_frames: 0` 이고, 문서도 "native curriculum이 처음부터 켜져 7.5 cm에서 1 cm·8°로 조인다"고 씁니다. 논문 Fig. 3a는 9B frame 동안 6 cm·50°로 고정입니다. 패치에 `tolerance_curriculum_start_frames` 옵션이 있어서 바꿀 수는 있습니다. 병렬 환경 24,576개, SAPG 6그룹입니다.	기본 실행으로는 Fig. 3a 수치를 재현할 수 없습니다.
D6	DexMachina 실험 코드가 없습니다. README의 벤치마크 표는 DeXtreme, SimToolReal, SPIDER 셋입니다("the four benchmarks"라고 쓰지만 표에는 셋뿐).	Fig. 5와 서론의 "+28%"는 재현할 수 없습니다.
D7	기준선 행동 공간에 논문에 없는 세부가 있습니다. Eigen-Space는 $q_\star$ = PCA 평균 + $E^{\top}c$이고, 행동 [−1, 1]을 계수의 P1/P99 범위로 비대칭 사상합니다. Eigen-Residual은 관절 목표에 "0.3 × (복원 자세 − 중심)"을 더합니다. Δ 변형은 초당 3 범위입니다.	Eq. 1과 다릅니다. Eigen-Space는 부분공간 안에서도 사람 범위 상자로 제한되고, Eigen-Residual의 eigen 기여는 0.3배로 줄어 있습니다. 기준선 결과를 해석할 때 필요합니다.

#	내용	영향
D8	무작위 방향 대조는 고정된 무작위 기저 **하나**이고, SPIDER에만 있습니다.	"사람 방향이 이득의 원천"이라는 결론의 근거가 무작위 기저 한 개와 SPIDER 한 벤치마크입니다. RL 벤치마크에서는 검증되지 않았습니다.
D9	README가 시간 상관 변형(AR(1), `correlation=0.94`, 지속 약 16 step)을 "기본보다 나을 수 있다"고 권합니다. 논문에는 결과가 없고, 공개 벤치마크는 모두 0(논문 설정)을 씁니다.	근거 없는 권고입니다. 쓰려면 직접 비교해야 합니다.

3. 수치 재계산 (`table_check.py`, `pca_check.py`, `fig_digitize.py`)

#	주장 (출처)	재계산	판정
C1	SimToolReal +34% (Sec. 1, Fig. 3a)	10.6 / 7.9 = +34.2% (seed 5)	✅
C2	DeXtreme +40% (Sec. 1)	12.3 / 8.8 = +39.8% (seed 5)	✅
C3	연장 학습 14.1 대 12.0 대 4.9 (Fig. 3b)	Joint 대비 2.88배, Eigen-Residual 대비 +17.5%. 방법마다 실행 1개. v1의 "larger gains in extended runs"는 v2 서론에서 빠짐	⚠️ 표본 1
C4	실물 DeXtreme +41%, 최장 +28% (Sec. 4.5.2)	8.45 → 11.90 = +40.8%, 32 → 41 = +28.1%. 방법마다 40회, 정책 1개, SD 미보고	✅ (검정 불가)
C5	DexMachina +28% (Sec. 1)	Fig. 5 판독, 과제 보상만: 평균 0.504 → 0.648 = +28.6%(평균의 비) / +26.2%(비의 평균). 커리큘럼 있음: +5.9%	✅ 과제 보상만에 한정
C6	SPIDER 평균 −16% (Sec. 1, Fig. 6)	여섯 손 평균 −16.05%. DoF와 Spearman 0.94 (p = 0.005)	✅
C7	Fig. 7b: ρ = 0.9에서 Joint 대비 −25%	0.066 / 0.089 = −25.8% (Fig. 6 Sharpa −25.4%와 반올림 범위에서 일치). ρ = 0.5 eigen만은 +41.6%	✅
C8	실물 SimToolReal 33% → 82% (Sec. 4.5.1)	Fig. 8의 과제별 평균(0.42/0.71, 0.34/0.92, 0.20/0.76, 0.36/0.90)의 평균이 0.33 → 0.8225. 물체-궤적 조합당 10회, 정책 1개, SD 미보고	✅ (검정 불가)
C9	k = 9(Sharpa), 6(XHand), 10(Allegro), Inspire 세 성분 90%	공개 자산에서 같은 k. Inspire k = 3 누적 93.4%	✅
C10	"이득은 상관 자체가 아니라 사람 방향에서" (Sec. 4.4, App. B.2)	SPIDER, 무작위 기저 1개로만 검증. toy에서도 같은 스펙트럼의 무작위 방향은 IID보다 나쁨(`03` §3.2)	⚠️ 범위 한정
C11	계산 규모	RL 실험 합계 약 9.4×10^{11} frame(그림에 나온 조건만). 연장 실행 1주 = 약 99k frame/s	참고

4. 구현 가능성 판단

4.1 공식 코드를 그대로 쓰는 경우

대상	요구 사항	판단
방법 패키지 (`packages/eigendexplore`, 약 430줄)	Python 3.10+, torch(README는 CPU wheel로 설치)	**바로 사용 가능.** 공개 손 6종은 PCA 자산 포함(MIT). 다른 손은 `from_pca`에 직접 만든 PCA를 넣으면 됨
CPU 예제 (`examples/ppo.py`, `sampling.py`)	torch CPU	가능
DeXtreme	Linux, NVIDIA GPU, IsaacGym(Python 3.8), 환경 16,384개, 실행당 2B frame	GPU 서버 필요

대상	요구 사항	판단
SimToolReal	Linux, NVIDIA GPU, Isaac Sim/IsaacLab 2.3.2, CUDA 12.6, 환경 24,576개. 9B frame은 약 25시간(1주 = 60B로 환산한 추정), 60B는 약 1주	GPU 서버 필요. Fig. 3a를 재현하려면 프로토콜 수정 필요(D5)
SPIDER	Linux, NVIDIA GPU, MuJoCo Warp 3.7, CUDA. 과제 데이터(OakInk2 10과제, 32 MB)는 저장소에 포함	GPU 서버 필요. 데이터 준비 부담은 작음
DexMachina, 데이터셋 비교, 다른 손의 기저	코드·코퍼스·fitting 파이프라인 없음	불가(D4, D6)

판단: 방법 자체는 바로 쓸 수 있는 형태로 공개되어 있습니다. 세 벤치마크도 고정 커밋과 패치로 재현 환경을 만들 수 있게 되어 있어서, 재현 정보는 좋은 편입니다. 다만 모두 NVIDIA GPU가 필요하고, SimToolReal은 기본 설정이 논문 서술과 달라서(D1, D5) 같은 수치를 기대하려면 설정부터 맞춰야 합니다.

4.2 이 맥(Apple M4 Pro, CUDA 없음)에서

단계	가능 여부	근거
공개 PCA 자산 검산, 초기 공분산·toy·그림 판독	✅ 완료	numpy, 설치 없음, 각 수 초
방법 패키지와 CPU 예제	가능할 것으로 보임 (torch 설치 승인 필요)	README가 CPU wheel 설치를 안내함. MPS에서 `LowRankMultivariateNormal`(Cholesky 등)이 도는지는 확인하지 않음
내 MuJoCo 과제에 EigenDEXplore 잡음을 붙인 표본 최적화	가능할 것으로 보임 (mujoco, torch 설치 승인 필요)	공개 Allegro·XHand 등 기저를 그대로 쓸 수 있음. SPIDER 벤치마크 자체는 MuJoCo Warp·CUDA 그래프를 써서 불가
DeXtreme, SimToolReal 학습	❌	IsaacGym·Isaac Sim은 CUDA 전용

4.3 아이디어만 다시 구현하는 경우

- **난이도: 매우 쉬움.** 핵심은 low-rank + 대각 Gaussian 하나입니다. torch에 `LowRankMultivariateNormal`이 있어서 수십 줄이면 되고, PPO의 log-prob·entropy·KL만 이 분포로 바꾸면 됩니다.
- **진짜 비용은 사전 분포입니다.** 내 손에 맞는 기저를 만들려면 사람 손 데이터(GRAB·ARCTIC 라이선스, EgoSuite 이용 조건 확인 필요)와 손별 리타겟이 필요합니다. 공개 코드에는 이 파이프라인이 없습니다(D4). 대안으로 텔레옵 로그의 관절 PCA를 쓸 수 있습니다.
- **비교할 때 반드시 맞출 것:** 탐색 총량(trΣ/d), 스펙트럼 지수, 그리고 같은 스펙트럼의 무작위 방향 대조군입니다. 공개 preset끼리도 이 셋이 달라서(D1-D3), 맞추지 않으면 결과를 구조의 효과로 읽을 수 없습니다.

02. 선행연구와 계보

번호 대신 저자-연도로 적었습니다(논문 참고문헌 형식). arXiv ID는 논문 참고문헌에 적힌 값을 옮겼습니다. 이 문서의 각 논문을 따로 열어 검증하지는 않았고, GitHub 정보는 2026-10-29에 `gh api`로 확인했습니다.

1. 계보도

```
[사람 손 synergy]  Santello et al. 1998 (자세 대부분을 소수 협응 패턴이 설명)
        │
[eigengrasp 계획]  Ciocarlie et al. 2007, Ciocarlie & Allen 2009
        │
[협응을 행동 공간에]  DFG (Agarwal 2023) · DextrAH-G (Lum 2024) · VAE synergy (Dimou 2021, 2024)
                      PCHands (Puang 2025) · Cross-embodiment grasping (Yuan 2025) · SAR
(Berg 2024) · LAMP (Yang 2026)
        │  한계: 저차원 행동은 미세 조정이 안 됨 (Lum 2024, Lee 2026 ADEPT, Gupta 2026a Grasp to
Act)
        ▼
[잔차로 표현 복구]  LUCID (Gupta 2026b, arXiv:2606.11628): eigen 계수 + 관절 잔차 = Eigen-Residual-
Δ
        │  질문: 이득이 "협응 명령"인가 "상관된 탐색"인가?
        ▼
  ★ EigenDEXplore: 행동은 관절 그대로, 사람 방향은 탐색 잡음에만

[구조화된 탐색 (사람 사전 없음)]  gSDE (Raffin 2021) · pink noise (Eberhard 2023) · colored noise
PPO (Hollenstein 2024)
                                  Lattice (Chiappa 2023, 정책이 만든 actuator 간 상관)
```

차별점 요약: 협응 사전 분포를 쓰는 선행연구는 그것을 **행동 표현**에 넣었습니다. 구조화된 탐색 연구는 **사람 사전 없이** 시간 상관이나 정책 유래 상관을 썼습니다. 이 논문은 둘을 갈라서 "사람 사전 분포를 탐색 공분산에만 넣는다"를 독립 변수로 만들었습니다. 비교 대상이 같은 저자 그룹의 선행 방법(DextrAH-G, Grasp to Act, LUCID)이고, 벤치마크도 같은 그룹 것(SimToolReal, DexMachina 공저)이 많습니다.

2. 구성 요소별 출처

역할	출처	이 논문에서의 쓰임
사람 자세 데이터	EgoSuite-Open100K (Lightwheel 2026), GRAB (Taheri 2020), ARCTIC (Fan 2023)	PCA 사전 분포. 공개 기저는 세 데이터 가중 혼합(`04` D4)
운동학 리타겟	ManipTrans (Li 2025, CVPR)	사람 손 → 로봇 관절. 공개 코드에는 리타겟·fitting 파이프라인이 없음
PPO	Schulman 2017, rl_games	log-prob·entropy·KL을 low-rank + diag Gaussian으로 계산
비구조 RL 벤치마크	SimToolReal (Kedia 2026, RSS), DeXtreme (Handa 2023, ICRA)	행동 공간 비교, 실물
레퍼런스 추적 RL	DexMachina (Mandi 2026, ICML)	커리큘럼 유무 비교
표본 기반 리타겟	SPIDER (Pan 2025, arXiv:2511.09484)	후보 섭동만 교체

역할	출처	이 논문에서의 쓰임
리타겟 과제 데이터	OakInk2 (Zhan 2024)	SPIDER 과제 10개
실물 자세 추정	FoundationPose (Wen 2024), AprilCube (Park & Agrawal 2026)	도구 waypoint, 큐브 자세
저차원 분포	`torch.distributions.LowRankMultivariateNormal`	Eq. 2를 그대로 구현(`04` §1)

3. 이 저장소의 다른 논문과의 관계

논문	관계
DexPolicy (`dexpolicy/`)	가장 가까운 주제입니다. 둘 다 dexterous RL의 **탐색 잡음**만 바꾸고 행동 표현은 그대로 둡니다. DexPolicy는 σ의 **크기**를 시간에 따라 줄이고, EigenDEXplore는 σ의 **모양**(공분산 방향)을 바꿉니다. DexPolicy 분석에서는 학습되는 σ가 짧은 예산에서 거의 움직이지 않았습니다(0.202 → 0.185). EigenDEXplore도 σ_q와 σ_e를 학습하지만 초기값에서 크게 벗어나지 않을 수 있습니다. 그렇다면 **초기 분산의 크기**가 결과를 좌우하므로, 공개 SimToolReal preset의 두 배 분산(`04` D1)은 더 중요한 교란이 됩니다. 두 방법은 함께 쓸 수 있습니다 (σ_q·σ_e를 따로 스케줄링).
DDPG (`ddpg/`)	DDPG의 OU 잡음은 **시간** 상관, EigenDEXplore는 **관절 간** 상관입니다. README가 권하는 AR(1) 변형(ρ = 0.94, 지속 약 16 step)은 두 축을 합친 것입니다. DDPG 분석에서 OU의 dt가 빠져 상관 시간이 구현마다 100배 달랐던 것처럼, 이 변형도 지속 길이를 제어 주기·행동 청크와 함께 적어야 비교할 수 있습니다.
Do as I Do (`do_as_i_do/`)	Do as I Do의 리타겟은 SPIDER 기반 annealed sampling MPC입니다. EigenDEXplore의 SPIDER 실험은 바로 그 표본 분포만 바꿔 cold start 비용을 25% 줄였으므로, Do as I Do 파이프라인에 그대로 꽂을 수 있는 후보입니다. 다만 Do as I Do의 이득 대부분은 warmup에서 나왔습니다(+41%p). 둘을 함께 비교한 실험은 없습니다.
FlashDexRetarget (`flashdexretarget/`)	DexMachina식 물체 보조 커리큘럼을 쓰는 대규모 off-policy 리타겟입니다. EigenDEXplore의 DexMachina 결과(커리큘럼과 보조 보상이 있으면 이득이 +6%로 줄어듦)를 보면, FlashDexRetarget처럼 보조 장치가 많은 파이프라인에서는 구조화 탐색의 여지가 작을 것으로 예상됩니다. FlashSAC는 tanh-Gaussian 대각 정책이라, Eq. 2를 쓰려면 tanh 이전의 low-rank 공분산과 엔트로피 목표를 다시 정해야 합니다.
AlphaNIST (`alphanist/`)	이진 보상과 성공률 기반 시작 상태 샘플링으로 탐색 문제를 푼 연구입니다. "보상 설계가 적을수록 이득이 크다"는 EigenDEXplore의 결론과 같은 방향의 문제의식입니다. 다만 AlphaNIST는 평행 그리퍼라 사람 손 사전 분포를 쓸 수 없습니다.
DexScrew (`dexscrew/`)	IsaacGym PPO로 손가락 회전 기술을 배웁니다. 기술 하나를 배우는 설정이라, 이 논문의 DeXtreme 실험(재배향, +40%)과 가장 비슷한 조건입니다.

4. 논문이 다루지 않았지만 관련 있는 연구 (내 판단)

주제	관련성	근거 수준
CMA-ES·natural ES	표본 분포의 공분산을 온라인으로 적응시키는 고전 방법입니다. EigenDEXplore는 공분산을 사람 데이터로 **미리 고정**한 셈이고, 저자들이 한계로 든 "학습 중 기저 적응"이 바로 이 방향입니다. SPIDER 같은 표본 최적화에서는 CMA식 공분산 갱신과 비교해야 공정합니다.	사전 지식, 서지 확인 필요
MPPI 공분산 학습 / 학습된 proposal	표본 MPC에서 proposal 분포를 학습하는 연구가 있습니다. 사람 사전 분포 대 과제에서 학습한 proposal 비교가 빠져 있습니다.	사전 지식
SAC류 엔트로피 목표	대각 Gaussian의 엔트로피 목표(−d)를 low-rank 공분산에 그대로 쓰면 의미가 달라집니다. off-policy로 옮길 때 다시 정해야 할 부분입니다.	내 해석

주제	관련성	근거 수준
Action chunking 정책의 초기 잡음	flow·diffusion 정책은 행동 청크를 Gaussian 잡음에서 생성합니다. 이 초기 잡음 공분산을 사람 협응 공분산으로 바꾸는 것은 같은 아이디어의 생성 모델 버전입니다.	내 해석

5. 관련 GitHub (2026-10-29 확인)

저장소	무엇인가	공식 여부	라이선스	★	마지막 push	확인한 커밋
hgupt3/eigendexplore	방법 패키지, 6개 손 PCA 자산, 벤치마크 어댑터·패치	공식	MIT	53	2026-10-11	`5d57180` (clone, 커밋 1개)
eigendexplore/eigendexplore.github.io	프로젝트 페이지	공식	표기 없음	0	2026-10-13	`5ee962b` (메타데이터만)
hgupt3/simtoolreal	SimToolReal 포크(Isaac Lab 포팅), 벤치마크가 고정해 씀	공식 벤치마크 원본의 포크	MIT	0	2026-09-21	`fddff74` (고정 커밋, 학습 설정 파일만 읽음)
tylerlum/simtoolreal	SimToolReal 공식 구현 (위 포크의 원본)	벤치마크 공식	MIT	424	2026-09-21	메타데이터만
isaac-sim/IsaacGymEnvs	DeXtreme 호스트	벤치마크 공식	기타 (표준 아님)	2,950	2024-10-26	`aeed298` (고정 커밋, PPO 설정 파일만 읽음)
facebookresearch/spider	SPIDER 호스트	벤치마크 공식	기타 (표준 아님)	590	2026-09-22	`7123845` (고정 커밋, 메타데이터만)
Denys88/rl_games	두 RL 벤치마크의 PPO 의존성	MIT	1,407	2026-10-29	DeXtreme는 1.6.5, SimToolReal은 vendored `da1c321` (메타데이터만)
mandizhao/dexmachina	DexMachina (공개 벤치마크에는 없음)	관련 공식	MIT	277	2026-03-23	메타데이터만
DLR-RM/stable-baselines3	gSDE (Raffin 2021) 구현	관련	MIT	13,943	2026-10-26	메타데이터만
martius-lab/pink-noise-rl	pink noise 탐색 (Eberhard 2023)	관련 공식	MIT	50	2025-11-20	메타데이터만
amathislab/lattice	Lattice 잠재 탐색 (Chiappa 2023)	관련 공식	MIT	49	2024-02-23	메타데이터만

한계: 4절의 서지와 CMA-ES 계열 비교는 내 사전 지식에 근거합니다. 블로그나 git에 올리기 전에 원문을 직접 확인해야 합니다.